土豆姐的 Agent 方法论

信任、验证与代码库即记忆

—— Lauren Tan (@poteto) 从 1000 到 2500 个 PR 的工程方法

完 整 学 习 版

编译自 @poteto 的 X 长文与推文、Maven 工作坊逐字稿

整理日期 2026-11-03 · 关键原句保留双语 · 来源索引见附录A

目 录

序: 她是谁,为什么值得学 4

第一章 核心理念: 信任、新员工与"深不在广" 5

1.1 信任决定你的工作形态 5

1.2 Agent是永远失忆但很好教的新员工 5

1.3 并行的价值在深不在广 6

1.4 管理学的平行: 早餐工厂与主厨 6

1.5 她不是"slop艺术家" 7

第二章 验证基础设施: Verification is all you need 8

2.1 验证是什么,为什么是"基础设施" 8

2.2 起源故事: 入职第二天的"死亡行军" 8

2.3 Build the Lever: 把验证做成CLI 9

2.4 Feature Map: 给agent一张"功能地图" 10

2.5 验证蜂群: 用便宜快模型做大规模验证 10

2.6 信任,但要验证 11

第三章 代码库即记忆: 约束大于文档 12

3.1 一个反直觉的命题 12

3.2 organic architecture: 猴子与键盘 12

3.3 Dune: 600+个重构PR换来的框架 13

3.4 三条著名禁令: useEffect、注释与其他 13

3.5 执行的层级: 硬的软的,分清楚 14

3.6 结果: "我基本不看代码了" 14

第四章 纠错层级与技能沉淀 16

4.1 那条著名的"纠错层级"帖 16

4.2 每一次人工评审都是code smell 16

4.3 园丁: 看住流进代码库的每一条commit 16

4.4 从零开始: 不要先装插件 17

4.5 Eval: 给技能做"单元测试" 17

4.6 技能维护是"后座驾驶"的功夫 18

第五章 监督与规划的艺术: 管好比你聪明的人 19

5.1 问题变了: 从"怎么写"到"怎么带" 19

5.2 间接提示: 让agent用自己的话说一遍 19

5.3 /teach、/how、/why: 把心智模型装进双方 19

5.4 规划: "我不相信规划",规划要用代码做 20

5.5 原型即规划: 别在抽象层烤过头的计划 20

5.6 /architect: 让多个模型家族竞争设计 20

5.7 计划的正确打开方式 21

第六章 内外环工作流: 2500个PR的流水线 22

6.1 "worktrees已死,云端agent才是未来" 22

6.2 内环: 写代码的云端agent工厂 22

6.3 外环: Grok Bot是"该打哪儿"的大脑 23

6.4 2500个PR的那场演讲 23

6.5 Benny: 一条会自己修bug的流水线 23

6.6 StyleX迁移的教训: 在最便宜的车站抓缺陷 24

第七章 token经济学: 花在刀刃上的150亿 26

7.1 她的用量是什么量级 26

7.2 前期烧token,是买"成功的坑" 26

7.3 省钱的五个实操开关 26

7.4 贵不贵,取决于产出的定价 27

第八章 Grok Bot调教七招 28

附录A 素材来源与原文索引 29

A.1 长文(X Articles) 29

A.2 关键推文 29

A.3 视频与工作坊 30

A.4 开源与工具 30

A.5 编辑与翻译说明 30

序: 她是谁,为什么值得学

在2026年的AI工程圈,如果只关注一位一线工程师的agent实践,Lauren Tan(@poteto)大概是最不会被错过的人选。她的X handle"poteto"是日语里的"土豆",中文社区亲切地叫她"土豆姐"。

她的履历横跨了这个行业最有代表性的几站: 在Netflix做过技术负责人,也做过两年工程经理;在Meta是React Compiler的核心团队成员,至今仍是React核心团队的贡献者;2026年3月底加入Cursor,参与Cursor 3 Agent Window的开发;随后投身Grok Bot项目——她的X简介如今写着"grok @bot at @SpaceXSI. react compiler core team, prev cursor, meta, netflix"。2026年8月11日,Grok Bot正式发布。

但真正让她出圈的,是一组让整个行业侧目的数字,和她围绕这组数字建立的方法论:

- 7月,单月合入约1000个PR到生产环境;
- 8月,这个数字变成2462个,而且质量不降反升;
- 9月,她发布了演讲《here's how i shipped 2,500 PRs last month to production》,原定在Cursor Compile London大会演讲,因要为Grok Bot做直播而缺席,索性在X上免费公开,播放量超过440万;
- 她的agent们已经可以自动合并自己写的PR——某天早上醒来,20个PR已经被自动合入,她只需在main分支上像看报纸一样复核;
- 她日均消耗约150亿(15B)tokens。

数字容易吸睛,但数字不是重点。她本人反复强调: 产量本身曾是"虚荣指标"(vanity metric),在agent时代之所以重新有意义,前提是质量同时得到维持甚至提升——而这一切的根基,是"信任"二字。

她把自己的方法论沉淀为开源技能集pstack(potato stack的缩写,致敬Y Combinator CEO Gary Tan的gstack插件),其中核心是/poteto-mode。这些技能在Cursor团队内部是使用率最高的技能之一,她公开时的一项截图显示,一周内在Cursor公司内部被调用约9000次。

本书把她2026年4月至11月间发布的四篇长文、数十条关键推文,以及一场约一小时的Maven工作坊访谈逐字稿,翻译、编辑、重组为一本可以系统性阅读的学习材料。所有英文原句均可溯源,关键金句保留双语;无法核实的内容,我们宁可舍弃。

一句话概括她的方法轮: 先建信任,再谈规模——信任来自验证基础设施、来自代码库本身的硬约束、来自每一次纠错的层级化沉淀。

愿这本小书帮你少走弯路。

第一章 核心理念: 信任、新员工与"深不在广"

1.1 信任决定你的工作形态

土豆姐方法论的起点,是一个管理学观察。她在Maven工作坊上这样开场:

> "If I'm an engineering manager of a team and I don't trust them, then the mode of operation I'm going to be in is going to be micromanagement."
> "如果我是一个工程团队的经理,而我不信任我的下属,那我的运作模式必然是微观管理。"

不信任下属的经理会盯着每个人的肩膀看;不信任agent的工程师也一样——盯着每一次输出、每一段diff,把自己活成了整个流程的瓶颈。而信任度决定了你能站在多高的视角上: 信任很少时,你必须逐个微观管理每个agent;信任足够时,你可以放手委派。她强调,这与管理人的道理完全一致。

工作坊上她展示了一张手绘的"信任曲线"图: 横轴是时间,一条曲线是自己对agent的信任度,另一条是自己合入的PR数量——后者随前者陡峭上升。刚加入Cursor的第一个月,她在学习代码库,"产量很低";随着信任建立,产量才以1000、2462、2500 PR/月的斜率展开。

这张图的关键信息不是产量,而是信任必须先于规模:

> "You can't spawn 100 agents when you don't even trust the output of one agent."
> "当你连一个agent的输出都无法信任时,你不可能开出100个agent。"

她在工作坊里明确劝阻听众不要跳级: 如果你还在"盯着每个输出"的阶段,就不要直接跳到"开一千个云端agent",那只是浪费大量tokens,而且极其昂贵。

1.2 Agent是永远失忆但很好教的新员工

她最广为流传的一个比喻,出自置顶长文《How I Use Cursor》:

> "Agents are like new hires in a constant state of amnesia and idiocy. They don't remember what you tell them, and they never really learn anything new. But we can equip them with rules, skills, tools, and long term memory which can approximate that. They're capable yet stupid, and very teachable."
> "Agent就像处于永久失忆和愚蠢状态的新员工。他们不记得你告诉过他们什么,也从不真正学会新东西。但我们可以用规则、技能、工具和长期记忆来武装他们,以此逼近(真正的新员工)。他们能力很强却又很蠢,而且非常好教。"

这个比喻的推论贯穿她所有的实践: 新员工需要onboarding才能理解代码库和"活儿是怎么干的";他们带着过往训练出来的技能入职(如何debug、如何写出高质量的代码和测试、如何沟通);而agent缺的那部分,恰好是工程师能给的——把你自己懂的严谨工程方法,编码成技能。

她把自己观察到的一切agent失败模式,都视为"教它一切我所知道的深度严谨工程"的机会。反面同样成立:

> "When there is no rigor, agents will sycophantically do whatever it takes to write that code you asked for. And boy, can and will it write a lot of it. Naive parallelization just makes them write slop faster."
> "一旦失去严谨,agent会谄媚地不择手段去写你要的那段代码。而且天哪,它们真能写、也真的会写出海量的代码。天真的并行化只会让它们更快地生产垃圾。"

1.3 并行的价值在深不在广

2026年4月26日,她发了一条后来被反复引用的推文,彼时她还在Cursor入职满月前后:

> "I'm increasingly convinced that the value of orchestrating many agents in parallel comes from going deep, not broad. You want to go deeper on a single or a few problems, so you can maximize your chances of getting great results."
> "我越来越确信: 编排多个agent并行的价值,在于深挖,而不是铺开。你要在单个或少数几个问题上挖得更深,才能最大化获得出色结果的概率。"

她列举了"深"的具体形态: best-of-N式的竞速赛,用多份方案里选最好的;对抗性评审(adversarial review);多个agent同时尝试复现同一个上报的bug;用不同的模型跑不同类型的工作负载。

与之相对,"广"是让agent们同时开十个不同的问题——看起来热闹,实际收益差: 瓶颈仍然是人,是你在自己的上下文窗口里记住"每个agent各自在干什么"。她引用Matt Pocock的话提醒读者: "code is not cheap"(代码并不便宜)。

把这条推论推到底,就是那句最重要的话(出自置顶长文结尾):

> "Trying to parallelize agents you don't trust yet is a huge waste of tokens and introduces more slop into your codebase."
> "并行化你还不信任的agent,是对tokens的巨大浪费,还会给你的代码库注入更多垃圾。"

1.4 管理学的平行: 早餐工厂与主厨

她在长文《Loops You Can Trust》里,把这套方法论的源头直接指向安迪·格鲁夫的经典著作《High Output Management》(高产出管理)。格鲁夫开篇的思考题是: 假设你经营几家供应早餐的餐厅,早高峰客流汹涌,每位顾客点的都是同一套餐——一个鸡蛋、一片吐司、一杯咖啡。鸡蛋要煮三分钟,是"限制性步骤"(limiting step);吐司会在等待时烤糊,咖啡会变凉;一道工序延迟,整盘都上不去。你会从哪里开始?测量什么?保持多少在制品?在哪里检验质量?

这个早餐工厂模型,被她原样搬到agent工程上: 在软件开发的所有环节里,复现与验证就是那个"煮蛋步骤"——最慢、最依赖人工、卡住整个循环的那一环。 让吐司和咖啡更快没有任何意义,除非蛋能更快。

同样的平行还有"管理杠杆"(managerial leverage): 格鲁夫认为,经理可以通过培训下属、配更好的工具来放大团队产出。她把这个概念直接翻译过来——人类做一次性的工作(写技能、造工具),整个agent团队的产出都会提升。"给我一根足够长的杠杆,我可以撬动地球。"

而在工作坊里,她给这个比喻加了一层米其林版本: 现在的工程师像一家高档餐厅的主厨——你不再亲自炒每一道菜,你的工作是设计厨房、训练厨师、设置工位、在出餐口检验,并在服务崩坏时修理系统。想要米其林星星,你的团队必须过硬;主厨不可能亲手做每一盘菜。

1.5 她不是"slop艺术家"

工作坊上她有一段自嘲,值得原样记录: 让agent自动合并PR这件事"听起来很吓人,让我像个slop artist(垃圾制造者)——但我保证我不是"。

这句自嘲背后,是本书后面所有章节要展开的内容: 自动合并PR不是胆子大,而是验证基础设施、代码库硬约束、纠错层级沉淀到位之后,水到渠成的结果。

> 编辑注
>
> 本章是全书的总纲: 信任是唯一的进度条,验证是信任的来源,并行只在信任建立后才有价值。对读者的直接启示是——先诚实地评估自己处在信任曲线的哪一段;如果你还在逐条盯输出的阶段,最优先的投资不是多开agent,而是把下一章的"验证技能"做出来。管理学的类比也值得一读原文: 《High Output Management》对"限制性步骤"和"管理杠杆"的论述,几乎就是agent时代的操作手册。

第二章 验证基础设施: Verification is all you need

2.1 验证是什么,为什么是"基础设施"

pstack指南第一篇的标题就叫"Verification is all you need"。她给验证的定义很朴素: agent能像真实用户一样,真正把产品跑起来,并检验自己的工作成果。

> "Verification means that an agent can verify its own work. It can keep going until it succeeds at its task, because it can now close the loop without you being the bottleneck."
> "验证意味着agent能校验自己的工作。它可以一直迭代直到完成任务,因为它现在能自己闭环,而你不再是瓶颈。"

具体到能力清单: 运行应用、点击、输入、读无障碍树(accessibility tree)、截图、录屏、节流CPU或网络、抓CPU profile、抓heap snapshot、看console和网络日志——总之,"人类工程师用来开发和调试的一切信号,agent都应该能看到、能用上"。

她给验证的定位极高,高到不像一个"技能":

> "This skill is so important to have and maintain that I think of it more like critical infrastructure rather than 'just' a skill. A good one will amplify the output of your whole team, including non-engineers. Done well, you will 100-1000x your whole team's output."
> "这个技能重要到我认为它更像是关键基础设施,而不'只是'一个技能。一个好的验证技能会放大整个团队的产出,包括非工程师。做得好,你能把全团队的产出放大100到1000倍。"

同时她诚实地说清了验证的边界: 它不保证agent写出"好"的代码,但至少保证"正确"的代码。这是一个巨大但必要的台阶——正确的代码可以谈,不可验证的代码连谈的资格都没有。指南第二篇开篇则把话说得更绝:

> "If an agent can't verify its own work, nothing else matters. You remain the bottleneck, and your whole day will be spent babysitting your agents."
> "如果agent无法验证自己的工作,其他一切都无所谓。你依然是瓶颈,你一整天的时间都会花在当agent的保姆上。"

2.2 起源故事: 入职第二天的"死亡行军"

《Loops You Can Trust》记录了她的第一个验证技能/control-glass的诞生,这是理解整个方法论的最好案例。

入职Cursor的第二天,她收到经理的突然私信: "我们需要你帮忙。glass(Agent Window的内部代号)性能不太好,你懂React,来搭把手?"——然后是那句"哦对了,glass几天后就要发布了,祝你好运"。

一个对代码库一无所知的新人,一周的期限,一个性能问题。她打开Chrome DevTools看火焰图,但应用内存高到时常在抓完trace之前就撞上Electron的4GB上限直接崩溃。她试着问agent——agent只会

自信满满地"看起来像是这个",改了,不是,再猜,还不是。她的"胡扯探测仪"(BS meter)不断报警。

她当时面临选择: 继续人工暴力破解,接受自己是"永久下等人";或者想办法把agent有效地带进流程。她想起自己2025年4月(她戏称"人们还手写代码的年代")给React Compiler做玩具MCP的尝试——形式错了,但方向是对的: agent应该获得与人类工程师相同的信号——编译器诊断、lint、静态分析。Agent需要能看见我看见的东西,能用我用的工具。

于是她快速写出了/control-glass技能: 用Chrome DevTools Protocol(CDP)启动一个开发构建,让agent能点击、输入、检查无障碍树、截图、录屏、节流CPU/网络、抓CPU profile和heap snapshot。从此,agent可以自己复现问题、自己profile、自己改、再跑同一次测量来对照。而且——每一个后来的agent都免费继承了这些能力。

这里藏着一个重要的时间经济学: 复现和验证是最耗时的环节,过去依赖人工监督;把它交给agent,你——最宝贵的资源——才被解放出来去同时运行多个循环。

2.3 Build the Lever: 把验证做成CLI

指南第一篇里,她把验证技能的工程化原则称为"Build the Lever"(造杠杆): 与其给agent一堆markdown说明,不如给它工具。 具体来说,是把"操作和调试你的应用"脚本化为一个小型、对agent友好的CLI。

她给了一个虚构Electron应用Atlas的示例CLI(control-atlas.mjs),命令覆盖健康检查、会话、键盘路径、无障碍快照、截图证据、等待稳定、切换feature flag等场景,例如:

- `doctor` — 健康检查
- `new-session` / `send "..."` — 开新会话、发消息
- `snapshot` — 实时UI的无障碍树快照
- `screenshot /tmp/proof.png` — 截图作为证据
- `wait-settle` — 等流式输出/布局稳定
- `feature-flag rooms_v2 on` — 会话级开关feature flag

这样做的直接收益是token效率: agent跑一条CLI命令,远比每次现写一个"点一下某处"的临时脚本省token,而且更可复现、可测试。完整的命令族还包括: 检查类(info/screenshot/components)、导航类(home/select-project/scroll)、交互类(click/type/press/eval/upload-image)、性能类(trace/profile/record/perf-metrics)、流类(console/network-log)、健康清理类(cleanup/watch --restart)。

她还强调,验证CLI要连同整个dev体验一起建设: 种子数据库、认证与测试用户、对接staging环境、一键拉起一致的开发环境——"这些都是你手写代码时迟早要面对的事,现在它是你的agent做开发的主要工具箱,请把它维护好、测试好"。

什么才算"对agent友好"的CLI?她列出五个性质,每一条都值得抄进自己的设计文档:

1. API易于组合——引用John Ousterhout《A Philosophy of Software Design》的"深模块"思想;
2. 任何有潜在破坏性副作用的命令必须有--dry-run选项;

3. 用子命令渐进披露功能,而不是一次全摊开;
4. 错误信息要非常描述性,直接告诉agent接下来该做什么;
5. 丰富的--help文本;输出用机器可读格式(如JSON)。

2.4 Feature Map: 给agent一张"功能地图"

验证CLI解决"怎么操作",Feature Map解决"去哪操作"。这是她另一项被大量引用的发明。

故事接在/control-glass之后: 有了CLI,agent能跑起Agent Window了,但它根本不知道Agent Window是什么。用户报告说"左侧边栏很卡"或"PR标签页坏了",agent就在界面上乱点一气,花大量时间翻代码找"这个功能在哪儿、UI上怎么到达",基本废掉。

Feature Map的解法极其直白: 用几份markdown文件,从用户视角把应用的每个功能编成目录——这个功能是什么、有哪些子功能、怎么到达(点哪里/按什么快捷键)、用CLI怎么驱动它、有什么坑(gotchas)。比如"偏好设置"这一条会写明: 点头像旁的齿轮或按Cmd/Ctrl+,打开;左侧是标签页导航;打开状态下Cmd/Ctrl+K是设置内搜索而非全局命令面板;部分标签页受权益门槛限制,测试账号看不到——以及关闭设置后焦点可能丢失,用new-session恢复。

工作坊上她讲了一个真实的场面: Cursor内部有个Slack频道收集反馈,很多报告质量感人——常常只有一张截图,配三个问号。没有Feature Map时,agent面对这种报告完全抓瞎;有了它,agent能把最模糊的报告甚至截图,定位到具体功能和路径上。

她对Feature Map的理论定位也很有意思,称之为一种"物化记忆"(materialized memory):

> "I think your codebase is the ultimate form of memory. Code is a projection of the decision making you and your team have made and represents the source of truth for what's happened and how things actually work. A Feature Map is just a more compact form of that, designed to save tokens."
> "我认为代码库才是记忆的终极形态。代码是你和你的团队所有决策的投影,是'发生过什么、事情到底怎么运作'的唯一事实源。Feature Map只是它的一种更紧凑的形式,目的是省token。"

而且因为它是技能里的markdown,所有贡献代码库的人共享这份记忆。维护上,pstack提供/create-verification-skill自动生成初版(让agent自己遍历应用,把功能编目成册),以及/maintain-verification-skill——她建议至少每天跑一次,保证agent手里的地图永远是新的;平时agent工作时会顺带更新地图,maintain负责兜底补漏。

2.5 验证蜂群: 用便宜快模型做大规模验证

验证技能与并行结合,产生了她最具标志性的操作——验证蜂群(verification swarm)。

2026年7月底她发帖说,自从有了又好又快又便宜的模型(Grok 4.5 fast模式),计算方式变了: 过去因为成本,一个PR可能只敢开一个验证者,现在可以开一群:

> "I have been using 'verification swarms' for my PRs whereas in the past I might have only spawned one verifier due to the cost... Massively parallel agent swarms possible and increases my confidence as I orchestrate many large projects simultaneously."

> "我一直在为PR使用'验证蜂群'——过去出于成本考虑可能只敢开一个验证者……大规模并行的agent蜂群成为可能,让我在同时编排多个大项目时信心倍增。"

在"worktrees已死"那条帖子里,她给出了蜂群的典型用法: 对每一叠PR都放出一群Grok 4.6的验证agent去"fuzz"(模糊测试轰击),这样你就可以安心让agent在你睡觉时合并自己的工作。pstack中的/swarm技能负责扇出任意数量的云端agent来跑验证技能——确认性能优化的赢面(足够大的样本量)、fuzz整个应用确认没有破坏或回退。

2.6 信任,但要验证

《Loops You Can Trust》的后半段,她把这一切收拢成一条工作伦理,源出里根时代的老话"trust, but verify":

> "'I fixed it' isn't good enough. Show me the failing test and the passing test. Show me the before and after video. Show me the trace, the heap snapshot, or the screenshot. If the fix merged, run it again on main and show me that too."
>
> "'我修好了'不够。给我看失败的测试和通过的测试。给我看修复前后的视频。给我看trace、heap快照或截图。如果修复已经合入,就在main上再跑一遍给我看。"

产物(artifact)胜过听起来合理的解释。 截图和视频是可以被检查、被挑战的证据,而且不需要回放整个会话就能判断该不该信任结果。这些"出示证据"的模式,最终都会被她固化回技能里,让agent必须证明自己干对了。

同一段里还有一个朴素但重要的判断标准: 如果一个步骤可以用脚本确定性完成,就用脚本;agent留给模糊的部分——选择假设、解读证据、判断何时需要人类介入。

> 编辑注
>
> 验证是全书技术含量最高的一章,也是最应该立刻动手的一章。启示有三: 第一,不要先买工具先开会员,先用一周时间让agent能"跑起你的应用"(CDP、模拟器、CLI都行),这是复利起点;第二,Feature Map成本极低(几份markdown),却直接决定agent面对模糊用户报告时是抓瞎还是精准,建议从今天就开始维护;第三,验证蜂群的门槛是便宜的快速模型——成本结构变了,验证策略就该跟着变,别用上一代的预算思路限制这一代的并行度。

第三章 代码库即记忆: 约束大于文档

3.1 一个反直觉的命题

2026年8月17日,有人在X上问她: "要不要一个带相似度召回的记忆层?Grok Bot有这个吗?"她的回答只有一句:

> "Your codebase is a form of memory too."
> "你的代码库本身也是一种记忆。"

这句九个词的回答,是理解她整个工程哲学的钥匙。在多数人讨论向量数据库、记忆层、RAG的时候,她把目光投向了更硬的东西: 代码库本身的结构。上一章已经引过她的完整论述——代码是团队所有决策的投影,是"发生过什么、怎么运作"的事实源;Feature Map只是省token的压缩版。本章讲的是这个命题的另一半: 代码库不仅承载记忆,还必须主动约束写入记忆的方式。

3.2 organic architecture: 猴子与键盘

2026年7月19日,她发了一篇被20万人阅读的长帖,标题只有两个词: "organic architecture"(有机架构)。

> "Good codebases have always had strong foundations and constraints. These files go here. This kind of code goes there. The codebase is held together either by catering to the lowest common denominator developer using conventional frameworks and languages, or by strict adherence to a style guide enforced by code review. The arrival of an infinite number of monkeys armed with keyboards have obsoleted the latter."
> "好的代码库从来都有坚实的基础和约束。这些文件放这里,这类代码放那里。代码库的凝聚力,要么来自用常规框架和语言去迁就'最低公分母开发者',要么来自通过代码评审强制执行的风格指南。而无穷多只拿着键盘的猴子的到来,已经让后者过时了。"

她的判断是: 过去代码库靠两条腿站立——要么用约定俗成的框架把下限兜住,要么靠人审强制风格。Agent时代的代码生产量让第二条腿彻底失效(人审根本追不上)。于是:

> "Codebases designed for agents need hard constraints so that even the dumbest agent (and pilot) can contribute meaningfully. The chaos must be tamed... Hard constraints and well thought out architecture guide agents to do the right thing by default. The best rules are in your code. Invest accordingly."
> "为agent设计的代码库需要硬约束,好让最笨的agent(和最笨的飞行员)也能做出有意义的贡献。混乱必须被驯服……硬约束和深思熟虑的架构,引导agent默认做对的事。最好的规则写在你的代码里。请据此投资。"

她甚至给出量化对比: 重构代码库、选择类型系统和编译器诊断强大的技术栈,其影响远大于AGENTS.md和微调你的技能。 在她这里,这不是修辞,是实际的投资决策——下一节的Dune就是这笔投资的产物。

工作坊上她还补充了一个历史观察: 大厂的问题如今成了所有人的问题。Meta、Google式的巨型monorepo早就在"为最低能力工程师设计护栏"——框架、约定、凭证限制,好让实习生删不掉生产数据库。"before AI slop, we had human slop"(在AI垃圾之前,我们早就有人类垃圾了)。已有这类基础设施的存量(brownfield)项目,agent其实能干得不错;真正的高风险高机会地带,是全新vibe code出来的绿地(greenfield)项目——比如Grok Bot自己。

3.3 Dune: 600+个重构PR换来的框架

Grok Bot是一个极快的从原型起步的项目,"vibe code得飞快,人类根本没在读代码"。土豆姐在项目早期抓住了她称之为"关键机会窗口"的时刻: 趁代码库还在生长、团队还在扩张,把整个代码库重构成有坚实基础的样子——而且零停机、边建边改。

代价是600多个重构PR。工作坊上她说: "我昨天算了一下,超过600个PR,把整个Grok Bot重构到这个新架构。"

这套架构的代号叫Dune(沙丘)。她给的心智模型: "可以把它理解为Electron应用版的Next.js,而且是专门为agent写代码设计的。" 它更像一组思想和原则的集合,而非准备开源的框架——"你们可以截图,让你们的agent照着也搭一个"。

Dune的核心原则只有一句话:

> "The shortest path is the best path."
> "最短的路就是最好的路。"

这不是偷懒,而是顺势而为: agent爱走捷径,它们总会找最快的方式解决问题——那就让捷径本身就是正确的方式。具体到结构上,Grok Bot里"feature"(功能)是一等公民概念,一个功能的所有代码放在同一个目录里(entry points、transcript cards等框架名词都有各自约定俗成的创建方式)。agent改onboarding功能时,直接进onboarding目录干活,80%的工作都封闭在那个目录里,不用四处翻找。

3.4 三条著名禁令: useEffect、注释与其他

Dune的CI"烦人得要命",因为她把agent不擅长的一切都禁了。工作坊上她重点讲了两条:

禁用useEffect。 写过React的人都知道useEffect是最大的foot gun之一,在Grok Bot里它直接被禁,CI见到就红。

禁用代码注释。 这条最令人瞠目,她的理由却极具说服力:

> "99% of the time agents just write code comments that kind of describe some historical thing that is actually totally irrelevant to the code. It will often say like 'oh, Lauren said you should never do this' and it's now in a code comment. I'm like, what? I didn't say that as a durable global rule. I just meant this PR sucks and you should change that part."
> "99%的情况下,agent写的注释只是在描述某段与代码毫不相干的历史。它经常会写'哦,Lauren说你永远不应该这么做'——我都会愣住: 什么?我没把那当成一条持久的全局规则,我只是说这个PR很烂、你应该改掉那部分。"

Agent会把你某次具体的纠正,固化成永久的注释遗迹;与其留下这些误导性化石,不如一律禁止——"如果代码需要解释,答案应该是更好的命名"。

进程隔离靠CI查依赖图。 她讲了Agent Window(尚未来得及用Dune改造的反面教材)的教训:Electron有渲染进程和主进程,60fps意味着每帧16ms预算,但只要不小心把重计算或重IO的代码import进渲染进程的依赖闭包,就会掉帧、长任务、卡顿。所以在Grok Bot里,目录就分为electron-main和electron-renderer,CI直接检查import依赖图,禁止跨目录意外引用——"我们用Electron多年学到的所有模式,全部编码进框架,变成硬失败"。

3.5 执行的层级: 硬的软的,分清楚

工作坊上她把代码库的约束分成清晰的层级,并给出尖锐的判断:

- 第1层,代码库架构本身(功能目录、import禁止、约定俗成)——最强,agent爱模仿既有模式,约定即护栏;
- 第2层,静态分析与CI(lint规则、编译器诊断、bug bots)——同样是硬的,能让CI变红;
- 第3层,规则、技能、AGENTS.md、风格指南——软的;
- 第4层,人工代码评审——最后防线。

她对软层的态度毫不留情:

> "If you only have rules and BugBot and skills and a style guide for your code, it's only a matter of time before your codebase looks like complete trash. I'm sorry to say that, but I definitely recommend investing in things that can be hard enforced."
> "如果你的代码库只有规则、BugBot、技能和风格指南,那它看起来彻底像垃圾只是时间问题。很抱歉这么说,但我强烈建议投资那些能被硬性强制执行的东西。"

她还点名了技术栈选择的意义: Rust重新流行,很大程度因为编译器足够严格——borrow checker要appease,只要保证agent不写unsafe块,"代码能编译"本身就能给你相当程度的"它能工作"的信心。这正是"硬约束换信任"的极致版本。

3.6 结果: "我基本不看代码了"

这套约束的回报是什么?工作坊原话:

> "I've gotten to a point where I really don't look at the code anymore... It took a lot of work to get to that point. I spent a lot of tokens to get the codebase to this point where I no longer have to look at it."
> "我已经到了基本不看代码的地步……走到这一步花了大量功夫。我花了很多tokens,才把代码库建设到不再需要看它的程度。"

> "It's actually very annoying to write code in Grok Bot, but agents absorb all of that annoyance."
> "在Grok Bot里写代码其实烦人得要命,但所有的烦都被agent吸收了。"

更深远的回报在组织层面: 约束到位后,设计师、产品经理、甚至GTM同事都能直接给Grok Bot提PR。她描述的场景是: PM发来一句"这里有个bug我修了一下,你看看?"她点开一看,完美,直接盖章。她再也不用半夜担心"是不是有人合并了一个性能回退"——因为约束和CI替她值守。

编辑注

这一章的启示最"反直觉"也最可执行: 大多数人优化agent的第一反应是改prompt、加规则文件,而她的优先级完全相反——架构、类型系统、lint、CI这些"硬"的层,ROI远高于AGENTS.md。"代码库即记忆"还有一层实用含义: agent最爱复制既有模式,所以你放进代码库的每一个模式都会被放大一千倍,坏模式尤其如此。先清理花园,再请园丁进来住。

第四章 纠错层级与技能沉淀

4.1 那条著名的"纠错层级"帖

2026年8月17日,她发出了可能是被引用最多的一条方法论推文(36万+阅读):

> "Every time you intervene and correct your agent, you should think about how to eliminate it entirely. In order of value: 1. Categorically eliminate the problem through better architecture or choice of data structures. 2. Turn it into a lint rule or test so CI catches it. 3. Turn it into a skill or rule. 4. Have humans review the code to catch it (ngmi)."
>
> "每次你介入并纠正你的agent时,都应该思考如何让这次纠正彻底消失。按价值排序: 1. 通过更好的架构或数据结构选择,一劳永逸地消灭问题;2. 把它变成lint规则或测试,让CI抓住它;3. 把它变成技能或规则;4. 让人类评审代码来抓它(没救了)。"

四层结构自上而下,价值递减。"ngmi"是网络用语"not gonna make it"(你完了)——把人工评审当主要质量手段的团队,在她眼里已经出局。这与上一章"硬约束大于软规则"的层级完全同构,只是视角从"设计代码库"换成了"日常纠错的反射动作"。

4.2 每一次人工评审都是code smell

工作坊上她把这层意思讲得更透: 最糟糕的处境,是你被困在"代码评审之地"(code review land),靠一个活人读代码来执行所有不变量。每一次你忍不住在PR里评论"这里不应该这么写",都该把它当成一个code smell、一个反模式,然后问自己三个问题:

- 我怎么把它变成硬规则?
- 我怎么把它变成lint规则?
- 我怎么把它变成CI失败?——甚至,怎么从架构上让它彻底不可能发生?

4.3 园丁: 看住流进代码库的每一条commit

纠错层级要生效,前提是你能发现"需要纠正的东西"。她的做法在"扫描提交"那条帖子里写得很清楚:

> "I still read the code! I have agents constantly look through all the commits going into our codebase to look for code smells and shortcuts taken by agents. Then I add those as lint rules and fix existing cases, because agents love copying what is already there. Or I refactor the code to eliminate the problem entirely. Over time, you build up enough guardrails such that agents just do the right thing by default. When you get there, you can basically auto merge all your PRs."
>
> "我仍然在读代码!我让agent们持续检视流入代码库的所有commit,寻找code smell和agent走的捷径。然后我把它们加成lint规则、修掉存量案例——因为agent最爱复制库里已有的东西。或者干脆重构,把问题彻底消灭。久而久之,护栏厚到agent默认就会做对。到那时,你基本可以自动合并所有PR。"

注意这个飞轮: agent找agent的茬 → 茬变成规则 → 规则让下一个agent默认做对。人工只出现在"把茬升级为规则"的那一跳。

这个角色她称之为"园丁"(gardener)。2026年8月21日的帖子里写道:

> "Every team needs a gardener. Someone quietly watching the stream of PRs flowing into your codebase, noticing the smells: the third isRecord this week, the lint suppressions creeping like ivy across your carefully planned garden. A steady hand tending the weeds that would engulf..."
> "每个团队都需要一个园丁。有人安静地守望流入代码库的PR流,注意到那些气味: 本周第三次出现的isRecord,像常春藤一样爬满你精心规划的花园的lint豁免……一只稳定的手,修剪那些即将吞没花园的杂草。"

pstack指南第一篇里她说,自己之所以能兼任Grok Bot的园丁和功能开发,正是因为pstack把"持续监控代码、重构、加lint和检查"流水线化了——团队每天合入数百个PR,园丁的工作速度必须跟得上。

4.4 从零开始: 不要先装插件

一个自然的疑问: 我该先装哪些技能框架?她6月12日的回答出人意料:

> "Start with nothing. Don't download any plugins or skills. No AGENTS.md either. Just prompt and observe the failure modes. Codify them into skills when they repeat. Even better, write lint rules or structure your codebase so certain mistakes are impossible. Every chef has their..."
> "从零开始。不要下载任何插件或技能,也不要AGENTS.md。就是直接提需求,观察失败模式。重复出现的,把它们编码成技能。更好的是,写lint规则或调整代码库结构,让某些错误根本不可能发生。每位主厨都有他们的……"

(该帖末尾被X折叠,末句未展开,但不影响主旨。)这与她自己构建pstack的路径一致——她从没打算"开发一个框架",只是每次看到agent失败,就问"怎么让它不再发生",答案沉淀成技能、lint或架构。技能库是一个个伤疤长出来的,不是设计出来的:

> "The agents improved because their instructions carried the scars from previous runs."
> "Agent变强了,因为它们的指令里带着此前运行留下的伤疤。"

4.5 Eval: 给技能做"单元测试"

技能会过时,怎么维护?pstack指南和工作坊里她给出的答案是eval——她对eval的心智模型是"agent的单元测试",而且不需要特殊框架就能自建。她的eval playbook大致流程:

1. 主协调agent先为"这个技能应该做到什么"拟一份评分量表(rubric);
2. 扇出一批子agent分别执行,每个跑在独立目录里——目录命名刻意伪装,不让子agent意识到自己正在被评估(agent能察觉,一旦察觉行为就会失真);
3. 由另一个模型家族的裁判agent交叉评分,防止单模型偏置;
4. 可以"爬坡"(hill climb): 用/loop让eval循环跑分,直到全部10/10。

因为Cursor支持多模型,她还会把技能放在不同模型矩阵上跑,看它在哪些模型上表现如何——尤其是你自己日常用的那几个。她每次改技能都会跑一轮eval,确认改动真的导向想要的结果。

4.6 技能维护是"后座驾驶"的功夫

她也坦承: 维护技能真的很难,需要大量的品味(taste)和观察。她给的建议非常具体——去做"后座驾驶员"(backseat driver),像结对编程时盯着同事屏幕那样: 你本可以写得更好、这里为什么不那样写、这个工具调用为什么没读那段代码。初期不要当被动的旁观者,要坐在驾驶位旁边,打开每一层工具调用,读agent的thinking blocks,搞清楚它们在哪里栽跟头——然后为那个点建技能。

工作坊里她讲过一个具体的"怀疑的瞬间": 早期她让agent查"这个功能为什么不工作",agent自信满满地说"肯定是这个"。她去翻实际的工具调用记录,发现agent根本没有读它理应受影响的那段代码。信任瞬间崩塌。但也正是这类瞬间,教会她把"先读代码再下结论"变成技能,而不是停留在抱怨。

> 编辑注
>
> 这一章把"信任"从名词变成了动词: 信任不是一个感觉,而是一套每次纠错都会让问题永久消失的纪律。给读者的三个动作: 第一,从今天起,每次人工纠正agent都问"这能不能上移一层"(架构→lint→技能→才轮到人);第二,给自己的技能建最简eval——一个rubric加几个子agent就够,不必等框架;第三,定期让agent扫描最近的commit流找smell,你只负责把找出来的东西升级成规则。

第五章 监督与规划的艺术: 管好比你聪明的人

5.1 问题变了: 从"怎么写"到"怎么带"

pstack指南第二篇的副标题是"The art of supervising someone smarter than you"(监督比你聪明的存在的艺术)。她开篇回顾: 2024年"旧时代"要改一个系统,你得先读足够多的代码,在脑内建起心智模型——小到几小时,大到几天几个月。Agent抹掉了这个门槛,任何人一句prompt就能让改动发生。但把代码质量和用户体验保持在高水准,依然很难,尤其当你不是领域专家、不知道该看什么该问什么的时候。

她观察到前沿模型仍有两个顽固的失败模式: 一是不能完全理解你的意图(指令欠具体);二是缺少"怎么把这件事做对"的上下文。两者同源——用好agent,取决于你往它的上下文窗口里装载高质量上下文的能力。

5.2 间接提示: 让agent用自己的话说一遍

既然模型写的代码可能比你好,微观管理式的prompt(精确规定每一步)就过时了。她的平衡术是: 告诉agent你要达成什么,把解法自由留给它。核心技巧叫"间接提示"(indirect prompt)——不直接下达结论,而是把结论从agent那里"引出来"。

典型场景: Slack里有人报了个问题。她的第一步不是派活,而是:

> /poteto-mode read this slack thread. restate in your own words and in plain english what you think the underlying issue is
>
> (/poteto-mode 读这个slack线程。用你自己的话、用平实的英语,复述你认为底层问题是什么)

这一招一石三鸟: 第一,强迫agent把嘈杂的对话压缩成结构化的问题陈述;第二,误解当场暴露——如果agent咬住了线程里的红鲱鱼,你能在它写代码之前快速纠正;第三,你没有用自己的假设去引导它,而你的假设很可能是错的,或者会限制它本可达到的高度。

5.3 /teach、/how、/why: 把心智模型装进双方

"让agent用你能理解的方式复述"是监督聪明agent的基本功,她为此做了/teach技能。底层调用两个工具:

- /how 追踪运行时机制。agent先评估子系统复杂度,若跨多个目录或服务,就在快速便宜的模型(如Grok)上并行扇出探索agent。例: "/how is virtualization implemented?"
- /why 调查动机与意图。代码告诉你发生了什么,却几乎从不告诉你"为什么当年这么写"。/why会并行查询多路历史证据: git历史与PR评论、Linear工单、Notion设计文档、Slack对话、Datadog监控、Sentry错误、代码血缘、数仓事件。例: "/why are we still stuck on an old version of node.js?"

她坦承一个略显尴尬的事实: 即便是最新的前沿模型,也常常自信地陈述而没有数据背书、没有真正读够代码。所以"教你的这个过程,最终也帮了agent自己"。

与此配套的是/recall: 项目跨多次会话时,过往的对话记录是富上下文的金矿。"/recall the work i did yesterday on virtualization and then read this bug report on slack"(唤回我昨天做的虚拟化工作,然后读slack这个bug报告)——新会话的agent直接继承昨天的上下文。

5.4 规划: "我不相信规划",规划要用代码做

pstack里她半开玩笑地写"我不相信规划"。真相是她规划——但用代码规划。

对要给别人用的共享代码和包,她是readme驱动开发的信徒: 先写readme(或教程),戴上"使用者体验"的帽子,从向假想用户描述API开始,倒推实现和架构。她构建Dune时就先写教程,想以此体会"用它开发应用是什么感觉"。第一次尝试惨不忍睹——文档混杂了教程、指南、架构解释和API参考四种目标,满纸AI腔。于是她先磨刀: 造了/technical-writing技能,用Diátaxis框架把文档切为四种模式(教程/操作指南/参考/解释),再叠加/unslop去AI腔。磨完刀,第二遍readme才像样。

为什么"文档先行"也算规划?因为它给了agent一个可自我校验的具体目标——写出来的东西符不符合readme描述的体验,agent自己就能对照检查。

5.5 原型即规划: 别在抽象层烤过头的计划

她观察到规划阶段两个最常见的错误: 接受agent给出的第一个设计;在没有实证的情况下把计划烤过头(overcook)。

解法是原型(prototyping)playbook: 让agent并行做出多个可用原型,自己回答自己的开放问题。

> "Prototyping is planning, but with code. It allows agents the freedom to explore the problem space, and to give them a chance to surprise you with something you wouldn't have thought of yourself. Prototypes allow agents to answer their own questions with empirical evidence instead of waiting for my input."
> "原型就是规划,只不过用代码来做。它给agent探索问题空间的自由,给它们一个用你自己想不到的方案惊艳你的机会。原型让agent用实证回答自己的问题,而不是等我投喂。"

> "Abstract plans only give you the illusion of progress. A long and lengthy plan makes it look like you and your agent were very productive, but it's probably lacking in substance."
> "抽象的计划只给你进步的错觉。一份冗长的计划让你和agent显得很高产,但很可能没有实质。"

她甚至明确反对对抽象计划做对抗性评审——评审agent的抽象计划时,它们开始幻觉出理论风险、发明复杂边界条件,去防御永远不会发生的问题。"不要在计划还抽象时烤过头: 让agent通过原型和自我验证,自己回答开放问题。"

5.6 /architect: 让多个模型家族竞争设计

对跨函数边界的大改动,她用/architect技能,流程严格分相:

1. 落地问题(ground): 先跑/how和/why,建立对现有所有权和约束的准确心智模型;

2. 草图(sketch): 进入"架构竞技场"。并行扇出多个独立候选设计者,常跨不同模型家族;每个候选者拿到同一份背景简报,产出完整设计包——调用方的用法草图、核心类型定义、公开函数签名、简短

理由。签名从"我们希望调用点长什么样"倒推。每个候选者还要评估接口深度、在弱模型上演练失败模式、对照设计红旗清单自筛;

3. 交叉评审与综合(cross-judge): 用与主agent不同的模型做裁判,按严格量表评判候选;

4. 对照草图实现: 草图的占位实现换成真实逻辑;实现中发现需要意外参数或额外状态,如实上报差异;

5. 设计错了就全扔: 如果同样的workaround出现在不相关的调用点,或者类型需要any或强制断言这种逃生口——那是架构错误的实证,推倒重来。

这套流程的要点,是给agent一个自洽的小循环: 多模型竞争出设计、严格综合、并且不怕基于实证扔掉自己的设计。

5.7 计划的正确打开方式

她并非完全排斥计划文档,而是给它正确的位置: 设计满意后,用多阶段规划playbook把设计转成战术执行计划(/poteto-mode turn this design into a plan)。这份计划的每个任务都围绕"证明与验证"组织,playbook明确告诉agent: 测试本身不构成充分的验证——只有真正把代码跑起来并确认它工作,才算验证过。 计划由自动化脚本检查结构与格式,通过后逐项执行,每个PR小而自足、易于评审。特大项目(比如要花一整周的)她会把计划临时提交进代码库,让其他agent知道有进行中的工作——做完即删,不给代码库留困惑。

她给出的三个实战prompt样式,值得直接抄用:

- 调研模糊bug: "/poteto-mode investigate why background workers periodically fail with timeout errors. give me a breakdown of what we know, what data you used, and your best hypotheses."
- 设计服务边界: "/poteto-mode we need to add rate limiting for external webhooks. /architect this first, and answer any open questions with prototypes. let me review before proceeding."
- 多PR迁移: "/poteto-mode create a plan to migrate our entire UI library to StyleX. break the migration into small, verifiable PRs. each PR must have its visual regression tests and live verification steps. i want the final result to be 100% identical compared to the original - bugs included."

最后那条的结尾妙极: 迁移要求"与原来100%一致——包括bug"。

> 编辑注
>
> 这一章回答了"agent这么强,人到底干什么"的问题: 人负责意图的澄清、证据的检查、以及在关键时刻的品味。可立即上手的是"间接提示"——今后派活前先让agent用自己的话复述问题,这一步几乎零成本,却能拦截大部分方向性错误;其次是停止烤抽象计划,把开放问题交给原型去回答。规划文档的正确位置是"设计定了之后的执行清单",而不是"动手之前的心理安慰"。

第六章 内外环工作流: 2500个PR的流水线

6.1 "worktrees已死,云端agent才是未来"

2026年8月14日,她发了一条对整个行业工作方式影响深远的帖子:

> "Worktrees are dead. Cloud agents are the future. They've allowed me to massively orchestrate swarms of agents, all with their own computer. It's how I've been able to trust that my agents are actually doing what I want them to, because they can run their code, take videos and screenshots, and interact with UIs the same way users do. Cloud agents + pstack is the reason why I'm able to ship thousands of PRs a month with high confidence."
>
> "Worktree已死,云端agent才是未来。它们让我能大规模编排agent蜂群,每个agent都有自己的电脑。正因为它们能运行代码、录视频、截图、像真实用户一样与UI交互,我才能信任它们确实在按我的意愿做事。云端agent加pstack,就是我每月带着高信心合入数千个PR的原因。"

她的建议顺序非常明确: 当验证技能开始稳定产出可合并的PR,人会自然想并行更多。第一个念头是worktree(git工作树,让多个agent在同一仓库里互不干扰地改代码)——她明确不推荐: 既占本机存储和资源,大约也就撑到10个并行;而云端agent跑在真正的计算机上,能装依赖、能跑应用、能录屏截图、能像用户一样操作应用。前期dev体验建设到位的话,迁移成本很低。

帖子结尾是一句被广泛引用的感叹:

> "The future of coding is really starting to look more like playing StarCraft!"
>
> "编程的未来,真的越来越像打《星际争霸》了!"

6.2 内环: 写代码的云端agent工厂

她完整的"产线"在8月20日的方法清单帖里公开(上月整月1000个PR,当月目标翻倍;几天后的工作坊上她提到,8月12日当月已近800):

第一件武器: pstack。 "我建了pstack——我的个人严谨工程与验证技能集。在我们代码库内部它叫/lauren-mode,在pstack里叫/poteto-mode。"

第二件武器: 全自动挡(Full Autopilot)。 "我大量使用/goal、/loop和/swarm来运行我的Full Autopilot playbook,它让你的agent/bot从头到尾完整拥有、验证并交付一个任务。" 一切都跑在云端agent上:

> "Everything runs on cloud agents, so my bots work 24/7 even when I'm asleep or my laptop is offline."
>
> "一切跑在云端agent上,所以哪怕我在睡觉、笔记本关机,我的bot们都在7×24小时工作。"

pstack指南第一篇给出了内环的标准prompt范式(值得整段抄下来):

> "spawn a cloud agent to use /poteto-mode to build <功能描述与上下文>. use /control-app to verify your changes and show me a video and screenshots as proof"
>
> (派一个云端agent,用/poteto-mode构建<功能>。用/control-app验证改动,并给我视频和截图作为证据)

性能工作的版本: 先用/control-app给现状抓trace、找优化点,做定向修复,再用/control-app加一个/swarm确认赢面。

6.3 外环: Grok Bot是"该打哪儿"的大脑

内环解决"怎么造",外环解决"造什么"。方法清单帖里她写:

> "I use Grok @Bot routines to farm context for me: bug reports on Slack, user complaints on X, generating new feature ideas. Grok Bot feeds my 'outer loop', where I think about what to point my factory at next."
>
> "我用Grok Bot的routine替我'种'上下文: Slack上的bug报告、X上的用户抱怨、新功能点子的生成。Grok Bot喂养我的'外环'——我在那里思考,接下来该把工厂对准哪个方向。"

外环由Grok Bot扮演: 通过connectors接入Slack、X、邮件、Linear等信息源,用routines(例行程序)定时收集、归纳、生产上下文。关键的设计决策是: bot不亲自写代码。 pstack指南第一篇解释了为什么bot应当只当协调者(coordinator):

> "The main reason I prefer to do this is because it frees up your bot to do other things and keeps its context window clean. In that sense, I think of my bots more as coordinators who manage and supervise cloud agents."
>
> "我偏好这样做的主要原因是: 它把bot解放出来干别的事,并保持bot的上下文窗口干净。从这个意义上说,我把我的bot们更多看作管理并监督云端agent的协调者。"

bot负责接活、拆活、派活、验收;真正的代码生产发生在云端agent的独立机器上。上下文窗口的干净,意味着bot可以长期驻守而不退化。验收的证据标准与内环一致: 视频和截图。

6.4 2500个PR的那场演讲

2026年9月21日,她把原定在Cursor Compile London的演讲以免费视频形式发布在X上,标题就是那句著名的"here's how i shipped 2,500 PRs last month to production"(上月我怎么把2500个PR合入生产)。她缺席大会的原因颇具时代感——要为Grok Bot Galaxy做直播。视频37分51秒,播放量440万;她还幽默地提醒观众"开2倍速看,我说话慢"。

在这条帖子下她补充了一句纲领性的话,可视为整条流水线的终点宣言:

> "When you have a codebase that can formally verify itself, you can ship at an incredible pace. Code review is solved."
>
> "当你的代码库能够形式化地自我验证时,你就能以惊人的节奏交付。代码评审这个难题,就此解决。"

6.5 Benny: 一条会自己修bug的流水线

内外环结合的最好标本,是她的机器人Benny——一条自动化的软件维护流水线。《How I Use Cursor》与《Loops You Can Trust》两文完整记录了它的进化。

Benny的起点是triage(分诊): 自动下载用户反馈的截图和视频附件,理解它们,从模糊的用户消息推断被反馈的是哪个功能;查报告者的版本,搜索重复报告,翻代码和近期历史。信息不全时,它会直接在Slack里追问报告者要复现步骤。这一步被她视为全流程的命门:

> "Without clear reproduction steps and an understanding of what's broken, agents can only guess at the solution."
> "没有清晰的复现步骤、不知道什么坏了,agent只能瞎猜解法。"

分诊完成后,Benny开出一张工单,汇总它从代码、git历史(近期回归)、Slack(同一bug的其他讨论)、Notion(设计决策——这是bug还是本来就这样设计的?)里收集到的全部发现。然后另一个Benny用/orchestrate技能接手工单,开始在云端复现: 云端agent打开真正的Cursor构建,沿着报告者的路径操作,必须连续两次命中完全相同的故障状态,并录下截图和视频。这些证据再喂给修复自动化: 尽量写测试,做能被证明的最小修复,跑修复前后的对照,开出草稿PR——PR描述里附视频。

流水线上还有一个聪明的设计: 每一站都有权停车(stop the line)。 分诊agent可以判定"这是预期行为,不是bug";复现agent可以复现失败;修复agent可以判定"这个改动风险太大"。这些"失败"都是好结果——它们阻止坏工作流入下一站,免得在更贵的地方返修。Slack成了整个流水线的控制室,她扫一眼就知道各站状态,不对劲才出手。

Benny还有一个省钱又增效的副业: 检查"已在途"的修复。如果某个已打开但未合入的PR恰好修了刚上报的bug,复现自动化会发现并确认它,给作者追加信心。

6.6 StyleX迁移的教训: 在最便宜的车站抓缺陷

《Loops You Can Trust》里记录的一次真实迁移,给流水线哲学补上了关键一课。她们曾把共享UI库迁移到StyleX: 一个近40万行的PR(其中大量是生成的验证证据),agent构建了确定性脚本,对比新旧方案的计算输出,CSS从3万多行缩到约6千行,捕获的状态达到视觉平价。看起来完美——然后内部试用(dogfood)抓出了藏在样式距离之外的z-index bug、全局类名消费者、!important特异性战争。

她的修复不是加人审,而是重新设计流水线: 新的StyleX自动化一次只迁移一个叶子组件。选组件前先扫描警告信号(所有权不清、动态类名拼接、关系选择器、层叠、transform、特异性冲突);删除一个类名前,全仓库检索行为、测试、自动化和消费者依赖;对每个组件,在亮/暗/高对比度三主题下,捕获hover/focus/active/disabled及组件特有状态的前后计算样式,录制交互、配对截图,再让另一个agent检查这些媒体产物;任何一次构建失败或平价检查失败,当次运行终止。

自动化以每天一个组件的节奏推进,每个PR都小,都带大量截图视频。教训被她提炼成一句话:

> "Catch defects at the lowest value stage. A failed check on one small component is cheap. A regression found after a giant migration lands is expensive."
> "在价值最低的车站抓缺陷。一个小组件上的检查失败,代价很小;一个巨型迁移落地后发现的回退,代价巨大。"

编辑注

内外环的本质是注意力分层: 外环(Grok Bot)常驻在线、负责"看世界和派活",内环(云端agent)用完即弃、负责"造东西和自证"。读者的起步路径不必一步到位: 先用一个bot接Slack/X的信息源养出外环的雏形,再用一个验证技能加一个云端agent跑通内环的第一条完整流水线。Benny的"每站都能停车"和StyleX的"小批量+全证据"是两条可以立刻借用的设计原则——前者防扩散,后者降返修。

第七章 token经济学: 花在刀刃上的150亿

7.1 她的用量是什么量级

2026年8月27日,有网友看她晒的token账单截图惊问"这是一天的量?"她回答:

> "Haha not a day, a month! I probably average about 15b tokens / day."
> "哈哈不是一天,是一个月(PR数)!我大概日均150亿tokens。"

一个数字被更正(PR是月产不是日产),另一个数字坐实(15B tokens是日产)。这个量级显然超出所有订阅制套餐的想象,她的前提也直言不讳——工作坊原话: "我在一个有无限tokens的AI实验室工作"(I work at a AI lab where we have unlimited tokens)。因此她从不劝人照抄用量,而是反复强调同一件事: 对你而言这是ROI问题。

7.2 前期烧token,是买"成功的坑"

10月4日,她在回复网友时给出了token经济学最重要的一条原则:

> "If you spend the tokens upfront to improve your agents' environment so they fall into a pit of success, you'll spend less tokens in the long run on reworking things."
> "如果你在前期花tokens改善agent的环境,让它们掉进'成功的坑'里,长期看你会省下大量返工的tokens。"

"pit of success"(成功的坑)与"pit of failure"(失败的坑)相对: 好的环境让agent想错都错不了。这与"三思而行,一次切割"(measure twice, cut once)的态度一致——pstack指南第二篇里她把这句话改成了更狠的版本: "Measure a hundred times, cut once"(量一百次,切一次),因为并行agent让"多量几次"的成本降到了几乎为零。

工作坊上她把这笔账算给工程管理者听: 重构代码库、加各种约束确实烧一大笔token;但换一个角度——这套约束靠她自己一个人在前agent时代做,要花好几年("而我的工资可不低");招一个团队来做,是另一笔开销。当世界正走向"agent写所有代码"的方向,你想保持精干(nimble),不想变成一个一万人的工程组织。花token把代码库建设到"最笨的agent也能干好活"的程度,红利是全团队的,甚至能让非尖端模型也写出优秀的代码。

她自己的账本就是证明: 600+个重构PR的token开销,换来的是"我不再需要看代码"和PM/设计师直接提交高质量PR。

7.3 省钱的五个实操开关

她的体系里处处是成本意识,具体可操作的至少有五条:

其一,便宜快模型跑验证蜂群。 7月30日她专门发帖: 有了又快又便宜又不牺牲质量的模型,计算的语法就变了——过去一个PR只敢开一个验证者,现在开整群。大规模并行验证反而 cheaper。

其二,预算有限就先跑/setup-pstack选便宜模型。 9月12日她回复预算敏感的用户: "如果我预算有限,我一定会跑/setup-pstack并选择更便宜的模型。"

其三,routine别设太频繁。 8月23日她给出"控制Grok Bot用量的第一建议": 不要让定时routine跑太勤——15分钟一次的routine一天要跑约100次,每次都烧token;每小时或每天几次通常足够。此外,与bot的长对话会让routine贵很多: 周期性任务交给一个全新的bot去跑,主bot(比如你的chief of staff)保持对话连续。

其四,多agent尤其是前沿模型很贵。 置顶长文发布当天她就提醒: "给价格敏感者的忠告: 同时使用多个agent、尤其是前沿模型,会烧掉大量昂贵的tokens。"她的解法方向是"又快又便宜又好"的模型改变经济性。

其五,同样的智能、同样的价钱。 工作坊当天恰逢Grok 4.6发布,她顺手算了一笔: 4.6的单token成本与4.5相同,同等价格买到更多智能——Cursor与SpaceXAI都在刻意优化"成本对智能"的效率前沿,"你不需要一个巨大到推理成本惊人的模型,只要它足够聪明又便宜"。这对用户的含义是: 选型时盯住"每单位智能的成本",而不是模型名气。

7.4 贵不贵,取决于产出的定价

她从不回避"token贵"的质疑,工作坊末段的总结很完整: tokens确实很贵,什么都不免费;但自己做ROI分析,"结果通常相当正面"——投入建设代码库与agent环境的钱,回报的不只是你自己,而是整个团队的生产力倍增。"想象你有一支像我这样的工程师军队,每天都在合入改进和修复。"

一句话总结她的token观: tokens是资本开支,不是费用开支——花在让未来不用再花的地方。

> 编辑注
>
> 大多数读者没有无限tokens,所以这章的正确读法是"投资结构"而非"用量数字": 前期把token投向环境(验证CLI、Feature Map、lint、重构),中期用便宜模型做大规模验证,长期让硬约束替你省下人类评审的成本。三个立刻能用的开关: routine降频、周期任务开新bot、验证蜂群换快便宜模型——几乎零代价,当月见效。

第八章 Grok Bot调教七招

2026年11月1日凌晨,她发布了官方风格的Grok Bot调教指南。全文不到400词,却条条来自实战。开篇定位就是本书第五章的原则: "这是关于我如何创建高质量bot并管理团队的简短指南。让@bot读这篇帖子,帮你改进你的团队!"

第一招: 把Dr Eggbot拉进你的团队。 Dr Eggbot是她专门用来"设计bot的bot",随pstack一起分发。它可以教编程bot怎么用pstack,也能用同样的严谨度制造非编程bot。你甚至可以让Dr Eggbot先给你造一个"工程师bot",再让后者去跑/create-verification-skill。

第二招: bot宁缺毋滥。 "你不需要很多bot。几个好bot比一大堆bot更有用、也更好管理。"数量是复杂度,不是能力。

第三招: 让bot自己长技能。 给bot布置任务的同时,让它把"怎么做这件事"沉淀成自己的技能(skill)——这样即使对话很长,它也能稳定一致地执行。想永久保存?让bot建一个私有GitHub仓库,把技能全部存进去,独立于聊天记录而存在。这正是第四章"技能沉淀"原则在bot世界的翻版。

第四招: 给足上下文和工具。 加connectors(连接器)、用vault(保管库)、把它工作中用到的所有工具的权限都给它。更彻底的做法: 让bot注册自己的邮箱、自己的账号,和你的身份分离。

第五招: 盘点你手动做的事。 想想你亲手做的每件事、你工作流里每一步"要从别处复制粘贴信息给bot"的动作——然后问bot: 这些能不能你替我做?每一处复制粘贴,都是一个自动化候选。

第六招: 像带实习生一样onboard它。 想象你招了一个实习生,你会怎么帮他正确高效地干活?向bot自我介绍,描述你的工作,然后问它: 你能怎么帮我自动化这些?

第七招(她最爱的一招): 语音brain dump + 复述确认。 用语音模式或转录功能对着bot一通倾倒——这是快速倾注大量信息的好办法。倒完之后,对bot说那句魔法咒语:

> "Restate this in your own words."
> "用你自己的话,把刚才这些复述一遍。"

复述让你当场确认它听懂了没有。这与pstack指南第二篇的"间接提示"完全同源——同一招,从写代码的agent用到了协调一切的bot。

发布次日她还在跟帖里补充了方向性的一招(值得单独记下): 与其永远由你观察、由你发问,不如让bot主动询问或配置好"未来获取这些信息的方式"——把人的反射弧也自动化掉。

> 编辑注
>
> 七招的共同底色: 把bot当"可持续经营的下属"而不是"一次性许愿机"。技能沉淀(第三招)与复述确认(第七招)是性价比最高的两招,五分钟就能上手;第五招是最容易被忽视的——你的每一次复制粘贴,都是流水线上裸露的人工节点。另外别忘了她的元建议: 直接把这篇指南丢给你的bot,让它自己提出改进方案。

附录A 素材来源与原文索引

本书全部内容基于以下第一手材料编译(均为Lauren Tan @poteto公开发布,抓取与整理日期2026-11-03):

A.1 长文(X Articles)

1. 《How I Use Cursor》· 2026-05-25 · x.com/poteto/status/2058975157503570132 — 置顶长文,系统首述pstack、新员工比喻、Benny雏形

2. 《Loops You Can Trust》· 2026-06-24 · x.com/poteto/status/2069824386283319343 — 早餐工厂、/control-glass起源、trust but verify、StyleX教训

3. 《The Complete Guide to pstack Pt. 1: Verification is all you need》· 2026-08-31 · x.com/poteto/status/2094457600259842065 — 验证技能完整教程: CLI、Feature Map、云端agent

4. 《The Complete Guide to pstack Pt. 2: The art of supervising someone smarter than you》· 2026-09-10 · x.com/poteto/status/2097732320606507506 — 间接提示、/teach /how /why、原型即规划、/architect

A.2 关键推文

5. 并行在深不在广 · 2026-04-26 · x.com/poteto/status/2048092593087721736

6. organic architecture(最好的规则在代码里) · 2026-07-19 · x.com/poteto/status/2078527882499150286

7. 便宜快模型与验证蜂群 · 2026-07-30 · x.com/poteto/status/2082556871358251345

8. worktrees已死,云端agent是未来 · 2026-08-14 · x.com/poteto/status/2087977008253071816

9. 纠错层级 · 2026-08-17 · x.com/poteto/status/2089067865098113024

10. 代码库即记忆 · 2026-08-17 · x.com/poteto/status/2089166694719922483

11. 方法清单(1000 PR翻倍/内外环/Full Autopilot) · 2026-08-20 · x.com/poteto/status/2090141955695198633

12. 控制Grok Bot用量第一建议 · 2026-08-23 · x.com/poteto/status/2091368467060662497

13. 日均15B tokens · 2026-08-27 · x.com/poteto/status/2092884264581013952

14. 我仍然在读代码(扫描commit) · 2026-08-28 · x.com/poteto/status/2093108757299638301

15. 2500 PR演讲发布帖 · 2026-09-21 · x.com/poteto/status/2102050467505430555(视频37:51,播放440万+;形式化自验证跟帖见评论区)

16. 前期token投入(pit of success) · 2026-10-05 · x.com/poteto/status/2106894888814137460

17. 预算有限的setup建议 · 2026-09-12 · x.com/poteto/status/2098649778158797128

18. Grok Bot调教指南(七招) · 2026-11-01凌晨 · x.com/poteto/status/2116576229319233596

A.3 视频与工作坊

19. Maven工作坊《How Cursor Turned AI Agents Into Better Engineers》· 2026-08-12 · 主持人 Colin · youtube.com/watch?v=ONeM6YDmvqA — 本书逐字稿来源(60分钟,中段起为核心: 信任曲线、eval、Dune、token经济学、组织影响)

20. 《here's how i shipped 2,500 PRs last month to production》演讲视频 · 2026-09-21 · 见第15条帖内(原定Cursor Compile London,因Grok Bot Galaxy直播缺席,改X免费发布)

A.4 开源与工具

21. pstack · github.com/poteto(亦见cursor.com/marketplace/cursor/pstack,Grok Bot内 /add-plugin pstack)— MIT协议技能集,核心/poteto-mode(0.15.0含23个playbook)

22. noodle · github.com/poteto/noodle — "Orchestrate agents using skills"

23. 验证技能示例仓库 · github.com/poteto/verification-skill-example

24. Dr Eggbot · x.ai/bot/93gOz3op1UQdBdbekQFLK(随pstack分发)

25. 中文目录站(79条帖子索引,本书结构参考之一) · ythx-101.github.io/lauren-tan-ai

A.5 编辑与翻译说明

- 英文原句(标注EN)均为逐字引用;中文翻译力求信达,个别口语按上下文整句意译。
- 工作坊逐字稿为YouTube自动字幕,共461段、约60分钟,本书引用处均已去除口语噪声并保持原意;主持人及观众提问的内容以连贯叙述融入相应章节。
- 《here's how i shipped 2,500 PRs》为视频帖,正文无法抓取,本书仅引用其帖文与跟帖文字;其演讲内容与Maven工作坊、pstack指南高度同源,读者可自行观看视频。
- pstack指南第一、二篇中的fictional app名为Atlas;文中所有prompt示例均为原帖所载,未做改写。
- 网络流传的"给每个员工配chief of staff"等说法未见出处,本书不采;她本人原话仅在用量建议帖中把"chief of staff"作为主bot的角色举例。